

Object-Oriented Programming

In This Edition

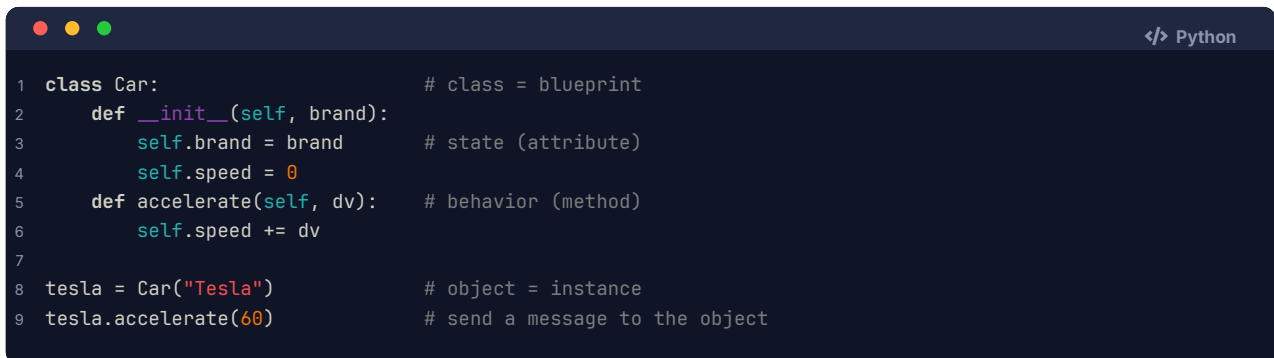
1	Overview --- What It Is	2
2	Why It Exists	2
3	Why FAANG Cares	3
4	Core Concepts	3
	4.1 Class vs Object vs Instance	3
	4.2 Members: instance vs static	3
	4.3 Access modifiers (visibility)	3
	4.4 Constructors & this/super	4
5	The Four Pillars	4
	5.1 1. Encapsulation --- <i>bundle + hide</i>	4
	5.2 2. Abstraction --- <i>expose what, hide how</i>	5
	5.3 3. Inheritance --- <i>is-a reuse</i>	5
	5.4 4. Polymorphism --- <i>one interface, many forms</i>	5
6	Object Relationships	6
7	Abstract Class vs Interface	6
8	SOLID Principles	7
9	Other Design Principles	8
10	Architecture / Diagrams	8
11	Real-World Examples	10
12	Real-Life Analogies	10
13	Memory Tricks / Mnemonics	11
14	Common Interview Questions	11
	14.1 Q1: What are the four pillars of OOP? Explain each in one line.	11
	14.2 Q2: Difference between an abstract class and an interface? When do you use each?	11
	14.3 Q3: Overloading vs overriding?	11
	14.4 Q4: Why prefer composition over inheritance?	12
	14.5 Q5: Explain the SOLID principles.	12
	14.6 Q6: What is the diamond problem and how is it resolved?	12
	14.7 Q7: How does runtime polymorphism actually work under the hood?	12
	14.8 Q8: What's the difference between association, aggregation, and composition?	12
	14.9 Q9: Can you achieve OOP without classes? Is JavaScript object-oriented?	13
	14.10 Q10: What is method hiding (vs overriding)?	13
15	Senior-Level Discussion Points	13
16	Typical Mistakes Candidates Make	13
17	How This Connects to Other Topics	14
18	FAANG Interview Tips	14
19	Revision Cheat Sheet	14

How to use this file: Read top-to-bottom for deep understanding. Jump to §Common Interview Questions + §Revision Cheat Sheet for last-minute revision. This is the conceptual foundation for **Low-Level Design** — master it before §06.

1 Overview --- What It Is

Object-Oriented Programming (OOP) is a programming paradigm that organizes software around **objects** — bundles of *state* (data) and *behavior* (methods) — rather than around functions and logic alone.

A **class** is a blueprint; an **object** is a concrete instance built from that blueprint.



```

1 class Car:                                # class = blueprint
2     def __init__(self, brand):
3         self.brand = brand                # state (attribute)
4         self.speed = 0
5     def accelerate(self, dv):              # behavior (method)
6         self.speed += dv
7
8 tesla = Car("Tesla")                      # object = instance
9 tesla.accelerate(60)                     # send a message to the object

```

OOP rests on four pillars — **A-PIE**: **A**bstraction, **P**olymorphism, **I**nheritance, **E**ncapsulation — and a body of design principles (SOLID, composition-over-inheritance) that keep object-oriented code changeable over time.

Paradigm	Organizing unit	State lives in	Example languages
Procedural	Functions	Global / passed around	C, Pascal
Object-Oriented	Objects (class instances)	Encapsulated in objects	Java, C++, Python, C#
Functional	Pure functions	Immutable values	Haskell, Clojure, (Scala)

2 Why It Exists

Before OOP, large programs were **procedural**: data structures and the functions operating on them were separate. As systems grew, this caused real pain:

- › **No clear ownership of data** — any function could mutate any global, so a bug could originate anywhere.
- › **Poor reuse** — copying logic led to divergent copies.
- › **Change ripple** — changing a data structure meant hunting every function that touched it.

OOP attacks **complexity** with four levers:

1. **Encapsulation** bundles data with the code that is allowed to touch it → a single place to enforce invariants.
2. **Abstraction** exposes a small, stable interface and hides the messy implementation → callers depend on *what*, not *how*.
3. **Inheritance & Polymorphism** let new behavior plug into old code without editing it → systems extend instead of mutate.

4. **Modeling** lets code mirror the domain (Order, Customer, Invoice), so the codebase reads like the business.

The goal of OOP is **managing change**: localizing it, and making extension cheaper than modification.

3 Why FAANG Cares

- › **LLD / machine-coding rounds** are *literally* OOP: "design a parking lot / elevator / splitwise" expects clean class hierarchies, the right relationships, and apt design patterns.
- › **Code quality signal** — naming, cohesion, encapsulation and SOLID adherence separate a senior from a junior in any coding round.
- › **Collaboration at scale** — large codebases survive only if modules expose stable interfaces and hide internals; OOP is the vocabulary teams use to reason about that.
- › **Language fluency** — interviewers probe abstract class vs interface, overloading vs overriding, virtual dispatch, and memory model — fast, correct answers show depth.

4 Core Concepts

4.1 Class vs Object vs Instance

- › **Class** — the template: fields + methods + constructors. Defines a *type*.
- › **Object** — a runtime entity created from a class, with its own copy of instance state.
- › **Reference** — a handle/pointer to an object (the variable), distinct from the object itself.

```

1 Car a = new Car("Tesla"); // 'a' is a reference; the object lives on the heap
2 Car b = a;                // b and a reference the SAME object (aliasing)
  
```

4.2 Members: instance vs static

	Instance member	Static (class) member
Belongs to	each object	the class itself
Copy	one per object	one shared copy
Access	obj.field	Class.field
Use for	per-object state	constants, counters, factory methods, utilities

4.3 Access modifiers (visibility)

Modifier	Same class	Same package	Subclass	World
private				
default (package)				
protected				
public				

Python has no enforced access control — convention is `_protected` and `__private` (name-mangled). C++ uses `private/protected/public` with friend exceptions.

4.4 Constructors & `this/super`

- › **Constructor** — special method that initializes a new object; can be overloaded.
- › **this** — reference to the current object (disambiguate fields, chain constructors).
- › **super** — reference to the parent (call parent constructor/method).
- › **Destructor / finalizer** — C++ `~Class()` is deterministic (RAII); Java/Python rely on GC, so use `try-with-resources` / context managers for cleanup.

5 The Four Pillars

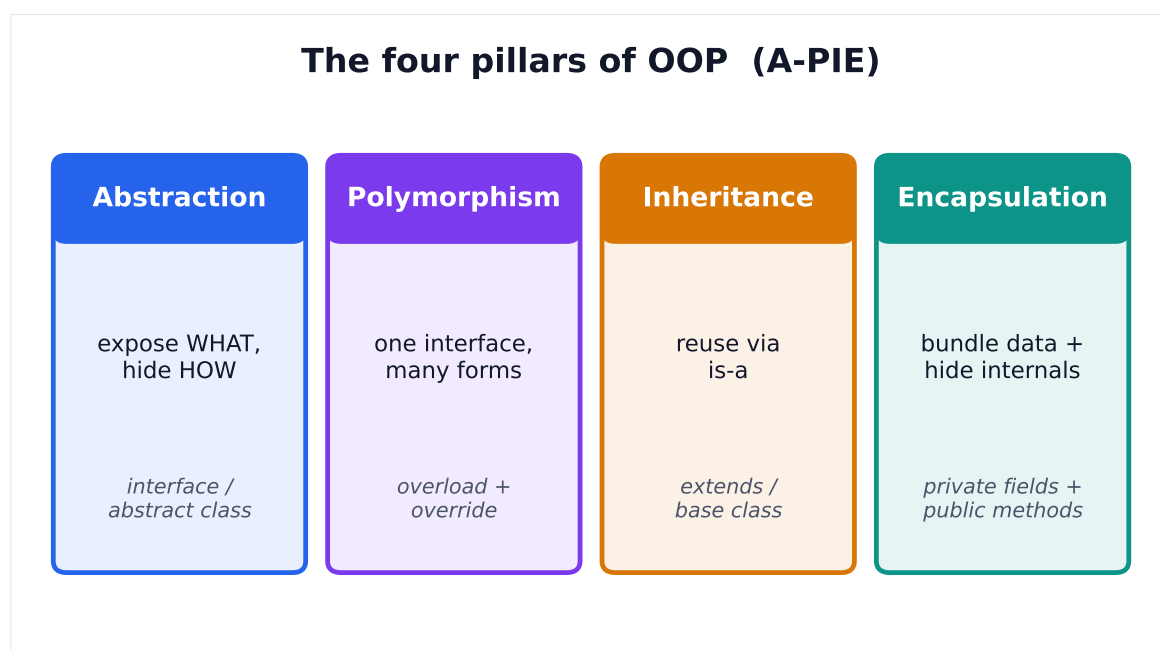


Figure 1. OOP rests on four pillars (A-PIE). Abstraction and encapsulation manage complexity; inheritance and polymorphism enable reuse and extension.

5.1 1. Encapsulation --- *bundle + hide*

Keep state **private** and expose behavior through a **public interface**, so the object controls its own invariants.

```
1 class BankAccount {
2     private double balance;           // hidden state
3     public void deposit(double amt) {
4         if (amt <= 0) throw new IllegalArgumentException();
5         balance += amt;               // invariant enforced in ONE place
6     }
7     public double getBalance() { return balance; } // read-only access
8 }
```

Why it matters: without encapsulation any code could set `balance = -1`. With it, every mutation goes through a guarded method. Encapsulation = **data hiding + a controlled API**.

5.2 2. Abstraction --- expose what, hide how

Model the essential behavior and suppress implementation detail. Achieved with **abstract classes** and **interfaces**.

```

1 from abc import ABC, abstractmethod
2 class PaymentMethod(ABC):
3     @abstractmethod
4     def pay(self, amount): ...      # WHAT, not HOW
5
6 class CreditCard(PaymentMethod):
7     def pay(self, amount): print(f"Charging card ${amount}")
8 class UPI(PaymentMethod):
9     def pay(self, amount): print(f"UPI debit ${amount}")

```

The checkout code calls `method.pay(amt)` and neither knows nor cares which concrete method it holds. *Abstraction is about the design (interface); encapsulation is about the implementation (hiding fields).*

5.3 3. Inheritance --- is-a reuse

A subclass **inherits** fields/methods from a superclass and may add or override them. Models an **is-a** relationship (Dog is-a Animal).

```

1 class Animal { void eat() { ... } }
2 class Dog extends Animal {           // Dog IS-A Animal
3     void bark() { ... }               // adds behavior
4     @Override void eat() { ... }      // overrides behavior
5 }

```

Types of inheritance: single, multilevel ($A \rightarrow B \rightarrow C$), hierarchical (one parent, many children), multiple (a class with two parents — Java/C# disallow for classes; C++/Python allow), hybrid.

Diamond problem — with multiple inheritance, if B and C both extend A and D extends both, which A does D get? C++ solves it with *virtual inheritance*; Python with the **MRO** (C3 linearization); Java sidesteps it by allowing multiple *interfaces* only.

Inheritance is the most *overused* pillar. Prefer **composition** unless there is a true, stable is-a relationship.

5.4 4. Polymorphism --- one interface, many forms

Kind	A.k.a.	Bound at	Mechanism
Compile-time	Static / ad-hoc	Compile time	Method overloading (same name, different signatures); operator overloading
Run-time	Dynamic / subtype	Run time	Method overriding + dynamic dispatch (virtual table)

```

1 List<Shape> shapes = List.of(new Circle(), new Square());
2 for (Shape s : shapes) s.draw();    // each calls ITS OWN draw() - runtime dispatch

```

The call `s.draw()` resolves to the actual object's method at runtime via a **vtable** (C++/Java) or `__dict__`/MRO lookup (Python). This is what lets you write code against `Shape` and have it work for shapes that don't exist yet.

6 Object Relationships

The "has-a" family — how objects collaborate. Strength increases left to right:

Dependency < Association < Aggregation < Composition < Inheritance
 "uses-a" "knows-a" "has-a (weak)" "owns-a (strong)" "is-a"

Relationship	UML	Lifetime coupling	Example
Dependency	dashed arrow	none (transient)	Order uses a PricingService passed to a method
Association	solid line	independent	Teacher ↔ Student
Aggregation	hollow diamond	independent (whole can die, part lives)	Team has Players (players exist without the team)
Composition	filled diamond	bound (part dies with whole)	House has Rooms (rooms gone if house gone)
Inheritance	hollow triangle	is-a	Car is-a Vehicle

```

1 // Aggregation: Engine passed in, outlives the Car
2 class Car { private Engine engine; Car(Engine e){ this.engine = e; } }
3
4 // Composition: Engine created & owned by the Car
5 class Car { private final Engine engine = new Engine(); }
  
```

Composition over inheritance: model behavior by *containing* collaborators rather than *extending* a base class. It avoids fragile hierarchies and the diamond problem, and lets you swap behavior at runtime (this is the heart of the Strategy pattern).

7 Abstract Class vs Interface

	Abstract class	Interface
Instantiable?	No	No
Methods	abstract and concrete	abstract (+ default/static in modern Java)
State / fields	yes (instance fields)	constants only (no instance state)
Multiple inheritance	one per class	many per class
Constructor	yes	no
Use when	shared base implementation + common state	a capability/contract many unrelated types can fulfill

	Abstract class	Interface
Models	"is-a"	"can-do"

Rule of thumb: *"Is it a kind of X?"* → abstract class. *"Can it do X?"* → interface (`Comparable`, `Serializable`, `Iterable`). Favor interfaces for flexibility; they keep the hierarchy shallow and enable composition.

8 SOLID Principles

Five principles that keep OO designs maintainable. Mnemonic: **SOLID**.



Figure 2. SOLID keeps object-oriented code changeable: each class focused (S), extended not edited (O), substitutable (L), with lean interfaces (I) and inverted dependencies (D).

- › **S — Single Responsibility (SRP):** a class should have one job. Invoice should not also format itself to PDF *and* email itself — split into `Invoice`, `InvoicePrinter`, `InvoiceMailer`.
- › **O — Open/Closed (OCP):** add new behavior by adding new code (new subclass/strategy), not by editing existing, tested code. Achieved via polymorphism. `if (type == ...)` chains are an OCP smell.
- › **L — Liskov Substitution (LSP):** anywhere a `Bird` is expected, a `Penguin` must work. If `Penguin.fly()` throws, the hierarchy is wrong — `fly()` doesn't belong on `Bird`. Subtypes must not strengthen preconditions or weaken postconditions.
- › **I — Interface Segregation (ISP):** don't force a class to implement methods it doesn't need. Split a fat `Machine{print;scan;fax}` into `Printer`, `Scanner`, `Fax`.
- › **D — Dependency Inversion (DIP):** high-level modules depend on **interfaces**, not concrete low-level classes. Inject a `Repository` interface, not a `MySQLRepository` — enables testing and swapping.


```
1 // DIP: high-level OrderService depends on the abstraction, not MySQL
2 class OrderService {
3     private final OrderRepository repo;           // interface
4     OrderService(OrderRepository repo) { this.repo = repo; } // injected
5 }
```

9 Other Design Principles

- › **DRY** — Don't Repeat Yourself: a single source of truth for every piece of knowledge.
- › **KISS** — Keep It Simple: the simplest design that works.
- › **YAGNI** — You Aren't Gonna Need It: don't build speculative generality.
- › **Composition over Inheritance** — prefer has-a to is-a.
- › **Law of Demeter** — "talk to friends, not strangers": `a.getB().getC().do()` (train wreck) couples you to internals; ask `a` to do it.
- › **Program to an interface, not an implementation** — the GoF mantra underpinning DIP.
- › **High cohesion, low coupling** — each module focused; modules minimally dependent.

Concept	Definition	Want
Cohesion	how focused a class is on one purpose	high
Coupling	how dependent modules are on each other	low

10 Architecture / Diagrams

UML class-relationship notation

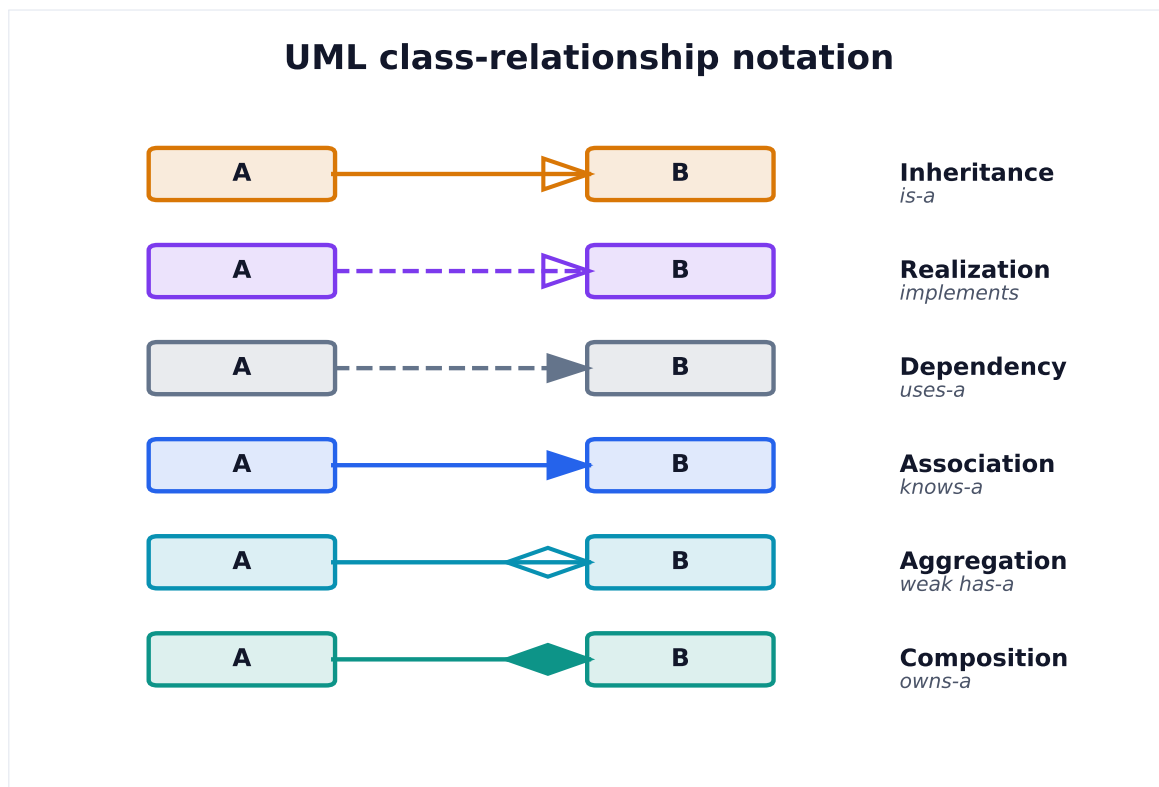


Figure 3. The UML vocabulary for class relationships, weakest to strongest: dependency, association, aggregation (hollow diamond), composition (filled diamond), inheritance (hollow triangle).

Inheritance vs Composition

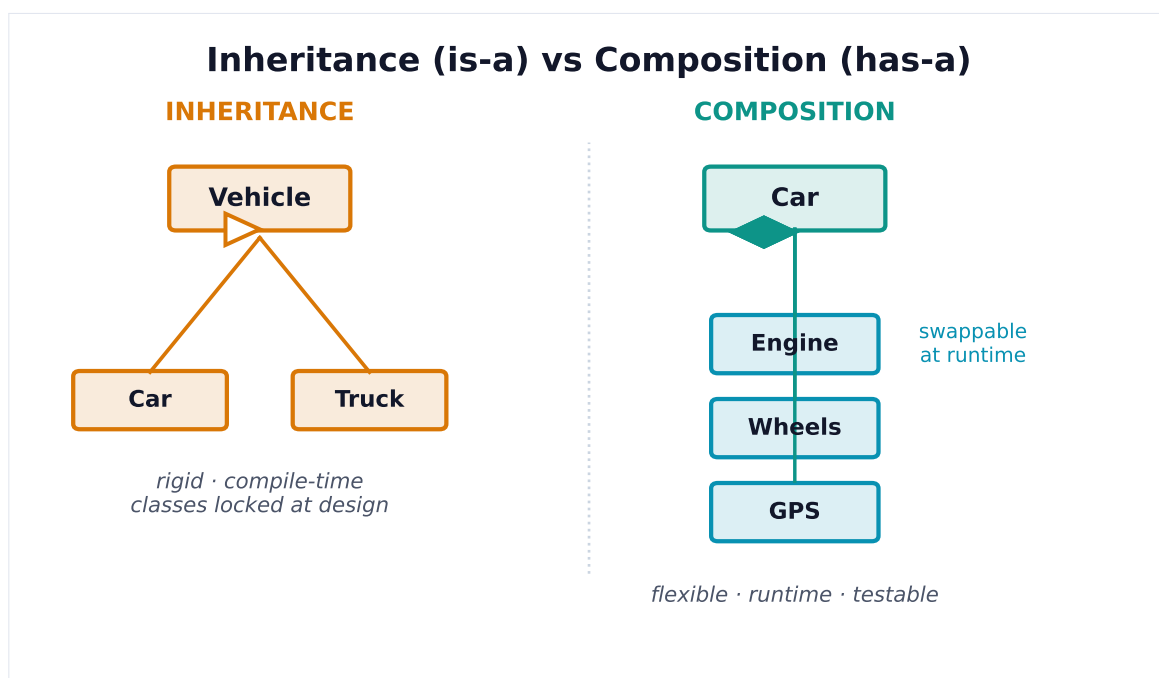


Figure 4. Inheritance hard-wires an is-a hierarchy at compile time; composition assembles behavior from swappable parts. Favor composition — it stays flexible and testable.

Runtime polymorphism (dynamic dispatch)

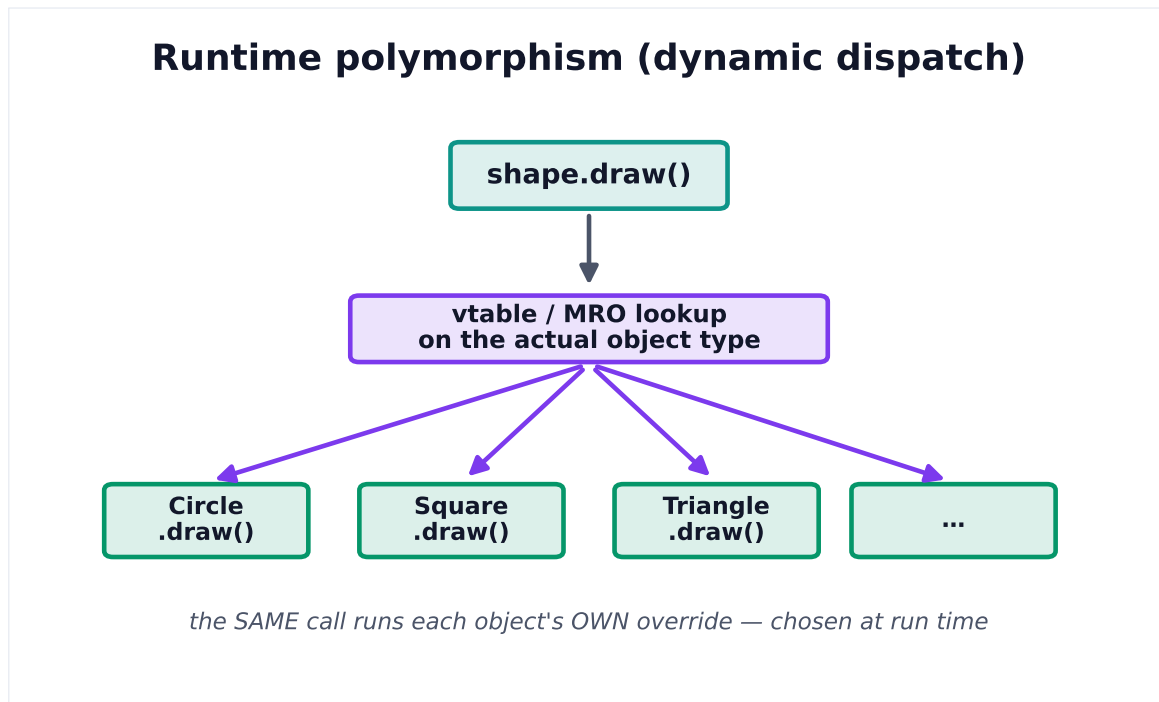


Figure 5. A virtual call resolves to the runtime object's method via a vtable (C++/Java) or MRO lookup (Python). This is what lets one line of code drive types written later.

11 Real-World Examples

- › **Java Collections Framework** — List, Set, Map are interfaces (abstraction); ArrayList, HashSet are implementations. You code to List, swap implementations freely (DIP/LSP in action).
- › **Payment gateways** — a PaymentMethod interface with CreditCard, UPI, PayPal strategies; check-out is closed for modification, open for new methods (OCP).
- › **GUI toolkits** — Button, Checkbox, Slider all extend Component and override render()/onClick() (inheritance + polymorphism).
- › **ORMs (Hibernate/Django)** — map classes ↔ tables; a Repository interface hides SQL (abstraction + DIP).
- › **Game engines** — GameObject with composed components (Transform, Renderer, Collider) — composition over inheritance, exactly to avoid a deep Entity tree.

12 Real-Life Analogies

- › **Class vs object** = an architectural **blueprint** vs the **houses** built from it. One blueprint, many houses, each with its own paint color (state).
- › **Encapsulation** = a **medicine capsule** — the active drug is sealed inside; you interact with the capsule, not the powder. Or an **ATM**: you press buttons (interface), the cash mechanism (internals) is hidden.
- › **Abstraction** = **driving a car** — you use the steering wheel and pedals; you don't think about fuel injection.
- › **Inheritance** = **family traits** — a child inherits eye color from a parent and adds their own.
- › **Polymorphism** = the word **"run"** — a person runs, an engine runs, a program runs: same verb, behavior depends on the subject.

13 Memory Tricks / Mnemonics

- › **A-PIE** → the four pillars: **A**bstraction, **P**olymorphism, **I**nheritance, **E**ncapsulation.
- › **SOLID** → the five principles (S/O/L/I/D).
- › **"is-a vs has-a"** → inheritance vs composition. When unsure, say "has-a" (compose).
- › **Overload = compile-time, Override = run-time** → "load later? no — overload is early (compile)".
- › **DRY-KISS-YAGNI** → the three "don't over-engineer" reminders.
- › **Cohesion HIGH, Coupling LOW** → "**H**igh **C**ohesion, **L**ow **C**oupling" = good design.

14 Common Interview Questions

14.1 Q1: What are the four pillars of OOP? Explain each in one line.

Model answer: **Encapsulation** — bundle data with the methods that guard it and hide internals behind a public API. **Abstraction** — expose essential behavior via interfaces/abstract classes and hide implementation. **Inheritance** — reuse and specialize via an is-a relationship. **Polymorphism** — one interface, many runtime forms (overriding) plus compile-time overloading.

Follow-ups:

- › "Encapsulation vs abstraction — aren't they the same?" → No: abstraction is a *design* concern (what interface to expose); encapsulation is an *implementation* concern (hiding fields, guarding invariants). Abstraction is about ignoring detail; encapsulation is about hiding it.

14.2 Q2: Difference between an abstract class and an interface? When do you use each?

Model answer: An abstract class can hold state and concrete methods and models an **is-a** relationship with shared implementation; a class extends only one. An interface is a **capability/contract** ("can-do") with (classically) no state; a class implements many. Use an abstract class when subclasses share code and state (`AbstractList`); use an interface to give unrelated types a common capability (`Comparable`, `Serializable`).

Follow-ups:

- › "Java 8 added default methods to interfaces — did that erase the difference?" → It narrowed it, but interfaces still can't hold instance state or constructors, and you can implement many. Choose interface for capability, abstract class for shared implementation + state.

14.3 Q3: Overloading vs overriding?

Model answer: **Overloading** = same method name, *different signatures*, resolved at **compile time** (static polymorphism), within one class. **Overriding** = same signature in a subclass, resolved at **run time** via dynamic dispatch (the actual object's method runs). Overriding requires an inheritance relationship; overloading does not.

Follow-ups:

- › "Can you overload by return type alone?" → No — the compiler can't disambiguate calls by return type; the parameter list must differ.
- › "What's covariant return type?" → An override may return a *subtype* of the parent method's return type.

14.4 Q4: Why prefer composition over inheritance?

Model answer: Inheritance is a tight, compile-time, white-box coupling: subclasses depend on superclass internals, deep hierarchies become fragile (the "fragile base class" problem), and it can't change at runtime. Composition is black-box: an object holds collaborators behind interfaces, enabling runtime swapping, easier testing (inject mocks), and no diamond problem. Use inheritance only for a genuine, stable is-a; otherwise compose.

Follow-ups:

- › "Give a concrete failure of inheritance." → Modeling Stack extends ArrayList exposes `add(index, e)` which violates stack semantics — an LSP break. Composition (Stack *has-a* list) hides it.

14.5 Q5: Explain the SOLID principles.

Model answer: **SRP** — one reason to change per class. **OCP** — extend by adding code, not editing existing. **LSP** — subtypes must be substitutable for their base without breaking callers. **ISP** — prefer many small interfaces over one fat one. **DIP** — depend on abstractions; inject concrete implementations. Together they reduce ripple effects and make code testable and extensible.

Follow-ups:

- › "Which SOLID principle does the Strategy pattern most embody?" → OCP (new strategies without touching the context) and DIP (context depends on the strategy interface).
- › "Give an LSP violation." → Square extends Rectangle where `setWidth` also changes height breaks code that sets width and height independently.

14.6 Q6: What is the diamond problem and how is it resolved?

Model answer: With multiple inheritance, if D inherits from both B and C which both inherit A, D has an ambiguous/duplicated A. C++ resolves it with **virtual inheritance** (one shared A). Python uses the **MRO** (C3 linearization) to define a deterministic order. Java/C# avoid it by forbidding multiple *class* inheritance, allowing multiple *interfaces* (no state to duplicate).

Follow-ups:

- › "If two Java interfaces define the same default method, what happens?" → Compile error; the implementing class must override and can call `Interface.super.method()`.

14.7 Q7: How does runtime polymorphism actually work under the hood?

Model answer: Each class with virtual methods has a **vtable** — an array of function pointers. Every object holds a hidden pointer (`vptr`) to its class's vtable. A virtual call indexes the vtable at runtime, so the actual object's override executes. Java does this for all non-final/non-static methods; C++ only for `virtual` ones; Python looks methods up dynamically via the instance `__dict__` then the MRO.

Follow-ups:

- › "Cost of virtual dispatch?" → One extra pointer indirection and a missed inlining/branch-prediction opportunity — usually negligible, occasionally hot-path relevant.

14.8 Q8: What's the difference between association, aggregation, and composition?

Model answer: All are "has-a". **Association** is a general "knows-a" with independent lifetimes (Teacher–Student). **Aggregation** is a weak whole-part where the part outlives the whole (Team–Player). **Composition** is a strong whole-part where the part's lifetime is bound to the whole (House–Room): destroy the house, the rooms go too.

Follow-ups:

- › "How do you tell aggregation vs composition in code?" → Composition usually *creates and owns* the part (`new Room()` inside, often `final`); aggregation *receives* the part via constructor/setter.

14.9 Q9: Can you achieve OOP without classes? Is JavaScript object-oriented?

Model answer: Yes — OOP needs objects, not necessarily classes. JavaScript was historically **prototype-based**: objects inherit directly from other objects via the prototype chain (`class` is syntactic sugar over prototypes). It supports encapsulation (closures/`#private`), inheritance (prototypes), and polymorphism (dynamic dispatch), so it's object-oriented, just not class-first.

14.10 Q10: What is method hiding (vs overriding)?

Model answer: Overriding applies to *instance* methods and is resolved by the object's runtime type. **Static methods can't be overridden** — if a subclass declares a static method with the same signature, it **hides** the parent's, and which runs is decided by the *reference* type at compile time, not the object. Same for fields (fields are never polymorphic).

15 Senior-Level Discussion Points

- › **Inheritance is coupling.** Senior engineers treat `extends` as a last resort. They reach for interfaces + composition, keeping hierarchies ≤ 2 levels deep.
- › **LSP is about behavior, not just signatures.** A subtype that compiles but changes the *contract* (throws where the base didn't, narrows accepted inputs) is a latent bug factory.
- › **Encapsulation at the module level.** Beyond fields, the principle scales to packages/services exposing narrow public APIs and hiding everything else — the same idea behind microservice boundaries and the Law of Demeter.
- › **OOP vs FP is a false binary.** Modern code mixes them: objects for boundaries/state, immutability and pure functions for logic. "Make illegal states unrepresentable" is a shared goal.
- › **Design patterns are vocabulary, not goals.** Knowing when *not* to apply a pattern (avoiding speculative `AbstractFactoryFactory`) is the senior signal.
- › **Value vs reference semantics, equality, and immutability.** Overriding `equals/hashCode` consistently, defensive copying, and `final`/immutable objects prevent whole classes of aliasing bugs and make objects safe to share across threads.

16 Typical Mistakes Candidates Make

- › **Reaching for inheritance first** to reuse code, building deep fragile trees instead of composing.
- › **Exposing fields publicly** (or auto-generating getters/setters for everything), defeating encapsulation — anemic objects that are just data bags.
- › **Confusing abstraction with encapsulation** in the definition question.
- › **Saying overloading is runtime** (it's compile-time) or claiming you can override static methods.
- › **God classes** — one class doing everything (SRP violation), low cohesion.
- › **Violating LSP silently** — `Square` extends `Rectangle`, `Penguin` extends `Bird` with `fly()`.
- › **Over-patterning** — forcing a design pattern where a simple function would do (YAGNI/KISS).
- › **Ignoring `equals/hashCode`** when putting objects in hash collections.
- › **Tight coupling to concretions** instead of injecting interfaces (DIP miss) — untestable code.

17 How This Connects to Other Topics

- › **Low-Level Design (§06)** — OOP is the raw material; LLD is OOP applied to a problem, plus design patterns and UML. Master this first.
- › **Design Patterns** — every GoF pattern is an application of these pillars/principles (Strategy = composition + OCP; Factory = abstraction + DIP; Decorator = composition over inheritance).
- › **System Design (§05)** — encapsulation and low coupling scale up to service boundaries and bounded contexts.
- › **Concurrency (§01)** — immutability and encapsulation are the cheapest path to thread-safety; shared mutable state is the enemy.
- › **Databases (§08)** — ORMs map objects to relational tables; the impedance mismatch (inheritance ↔ tables) is a classic OO/relational tension.
- › **Languages** — the abstract-class/interface, dispatch and memory rules differ across Java/C++/Python/Go (Go favors composition + interfaces, no inheritance at all).

18 FAANG Interview Tips

- › In an LLD round, **start by naming the nouns** (entities → classes) and verbs (behaviors → methods), then the relationships (is-a vs has-a). Narrate it.
- › When you choose extends, **justify the is-a out loud** — or, better, default to composition and say why.
- › Drop the principle by name when it applies: *"I'll inject the repository to honor DIP and keep this testable."* It signals seniority.
- › For "explain X vs Y" questions, give a **one-line definition each + when to use each + a concrete example** — that three-part structure reads as mastery.
- › If asked to extend a design, reach for **OCP**: add a new class/strategy rather than editing existing classes — and say so.
- › Keep hierarchies shallow; if you're drawing a 4-level tree on the whiteboard, pause and ask whether composition is cleaner.

19 Revision Cheat Sheet

Concept	One-liner
Class / Object	blueprint / instance built from it
Encapsulation	bundle data + hide it behind a guarded API
Abstraction	expose <i>what</i> (interface), hide <i>how</i>
Inheritance	is-a reuse via extends
Polymorphism	overloading (compile-time) + overriding (run-time)
Abstract class	is-a + shared state/impl, single
Interface	can-do capability, multiple
Association/Aggregation/Composition	knows-a / weak has-a / owned has-a
Composition > Inheritance	prefer has-a; flexible, runtime-swappable
SRP	one reason to change
OCP	extend, don't modify
LSP	subtypes substitutable for base

Concept	One-liner
ISP	small interfaces > fat ones
DIP	depend on abstractions; inject concretions
DRY / KISS / YAGNI	no duplication / keep simple / no speculation
Cohesion / Coupling	want HIGH / want LOW
Overload vs Override	compile-time signature vs run-time dispatch
Static method	hidden, not overridden; bound by reference type
Diamond problem	multiple inheritance ambiguity → virtual (C++) / MRO (Py) / interfaces (Java)

Pillars: A-PIE — Abstraction, Polymorphism, Inheritance, Encapsulation. **Principles:** SOLID + DRY/KISS/YAGNI + Composition-over-Inheritance + Law of Demeter. **Golden rule:** *favor composition, program to interfaces, keep things cohesive and loosely coupled.*